

What is a Constructor in Java?

A **constructor** is a special method used to **initialize an object**.

- It has the **same name as the class**
- It **does not have a return type**
- It is **automatically called when an object is created**

Job of Constructor

- Initialize object variables
- Allocate memory for object
- Perform setup when object is created

👉 Example:

```
class Test {  
    int x;  
  
    Test() {  
        x = 10;  
    }  
}
```

Types of Constructor

1) Default Constructor

- Provided by Java **automatically**
- Only if no constructor is written

```
class Test {  
    // Java provides default constructor internally  
}
```

2) User-Defined Constructor

- Created by programmer

```
class Test {  
    Test() {  
        System.out.println("User constructor");  
    }  
}
```

Types of User-Defined Constructor

1) Non-Argument Constructor

```
class Test {  
    Test() {  
        System.out.println("No argument constructor");  
    }  
}
```

2) Parameterized Constructor

```
class Test {  
    int x;  
  
    Test(int a) {  
        x = a;  
    }  
}
```

Rules of Constructor (Important)

- Name must be same as class name
- No return type (not even void)
- Called automatically when object is created
- Can be overloaded
- Cannot be static, final, or abstract
- Can have access modifiers (public, private, etc.)

Can User Call Constructor Manually?

Your example:

```
class Test{  
    public static void main(String[] args) {  
        Test t = new Test();  
        t.Test(); // ❌ invalid  
    }  
  
    Test(){  
        System.out.println("hii");  
    }  
}
```

❌ **This is WRONG**

- Constructor is **not a method**
- You **cannot call it like t.Test()**

✓ Correct:

```
Test t = new Test(); // constructor is called here
```

Can We Create Static Constructor?

✗ No, Java does NOT support static constructors

Why?

- Constructor is used for **object initialization**
- Static belongs to **class level**
- Constructor runs when **object is created**, not class loading

👉 Instead, Java uses **static block** for class-level initialization

Constructor Overloading

- Multiple constructors with different parameters

```
class Test {  
    Test() {  
        System.out.println("Default");  
    }  
  
    Test(int x) {  
        System.out.println("Parameterized: " + x);  
    }  
}
```

Non-Static Control Flow

This is VERY important for interviews.

Order of Execution:

1. Static variables

2. Static blocks
3. Main method
4. Object creation
5. Instance variables
6. Non-static block
7. Constructor

Example

```
class Test {  
  
    static int a = 10;  
  
    int x = 20;  
  
    {  
        System.out.println("Non-static block");  
    }  
  
    Test() {  
        System.out.println("Constructor");  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Main method");  
        Test t = new Test();  
    }  
}
```

Output

Main method
Non-static block
Constructor

Explanation

- Static part already executed during class loading
- main() runs
- Object created:
 - Instance variable initialized
 - Non-static block executes
 - Constructor executes

Final Quick Summary

- Constructor = object initialization
- No return type
- Cannot be static
- Automatically called
- Can be overloaded
- Cannot be called manually like a method
- Static block = class level init
- Constructor = object level init